TALH at 150M: Ablating a Hybrid MLA+SSM Architecture for Edge Language Model Inference on Apple Silicon

Christos Koutsiaris
Development Expert, SAP P&E, Cloud ERP, UX Foundation
christos.koutsiaris@sap.com

***Abstract*—We present a controlled ablation study of TALH (Adaptive Latent Hybrid), a sub-200M language model combining Multi-head Latent Attention (MLA) and a parallel State Space Model (SSM) branch. Five architectural variants are trained from scratch on FineWeb under identical compute budgets and benchmarked on an M3 MacBook. The study produces two concrete findings. First, the SSM and MLA components serve orthogonal roles: the SSM branch is the primary quality driver (SSM-only achieves PPL 239 vs. 263 for a standard Transformer baseline), while MLA drives inference efficiency rather than quality (MLA-only scores the worst perplexity at PPL 315, yet shows the best latency scaling). Second, MLA's KV-cache compression is directly observable as a latency property on unoptimised Apple Silicon: the MLA-only variant's time-to-first-token grows by only 1.32× for a fourfold context increase (512 to 2,048 tokens), while all other variants grow 2.7–5.5×. We also provide a controlled negative result: a dense feed-forward hybrid outperforms the ternary MoE variant at this scale, adding memory overhead without quality benefit.**

***Index Terms*—edge inference, language models, Apple Silicon, state space models, mixture of experts, multi-head latent attention, ablation study**

I. Introduction

Running capable language models locally, without sending data to a remote server, is a practical goal that standard Transformer architectures make difficult. Two bottlenecks dominate. First, the key-value (KV) attention cache grows linearly with context length in terms of entries but the cost of computing attention over that cache grows super-linearly, making long prompts increasingly slow. Second, even at moderate context lengths, the total memory footprint of a competitive model often exceeds what commodity hardware provides.

Several architectural directions have been proposed to address one or both bottlenecks. Grouped-query and multi-query attention reduce KV cache size by sharing keys and values across heads [8]. State space models (SSMs) such as Mamba replace attention entirely with a recurrent state update that has constant memory in context length [2]. Hybrid architectures interleave or combine attention and SSM layers to get the complementary strengths of both [3]–[5]. More recently, DeepSeek-V2 introduced Multi-head Latent Attention (MLA), which compresses the KV cache via a low-rank joint factorisation of keys and values [1], and Hymba proposed parallel within-layer attention+SSM hybrid heads specifically targeting small, on-device models [6]. Zebra-Llama explicitly combined MLA with SSM layers and reported large KV-cache reductions at 1–8B scale [7].

TALH sits in this same design space: it combines MLA with a parallel SSM branch and optional sparse expert routing, and targets sub-200M edge deployment on Apple Silicon. Rather than claiming a novel architecture, we contribute a **controlled ablation study** that asks a question this space has not cleanly answered at this scale: when MLA and SSM run in parallel within the same layer, do they serve the same objective or different ones? Concretely:

- Which component, MLA or SSM, is the primary quality driver in the hybrid?
- Does MLA's KV-cache compression translate into measurably better inference latency scaling on real Apple Silicon hardware?
- Does ternary MoE routing improve over a plain dense feed-forward layer at this scale?
- Under equal compute, how does the hybrid compare against a standard Transformer of comparable size?

We answer all four questions with a five-way ablation trained on FineWeb, followed by inference benchmarks on a MacBook M3. All architectural comparisons are made on a compute-matched basis: same training steps, same data, same curriculum for every variant. We are explicit about where results are not fully matched.

II. Related Work

KV-cache reduction. Grouped-query attention (GQA) and multi-query attention (MQA) reduce cache size by sharing K/V projections across heads and are now standard in production models [8]. DeepSeek-V2's MLA takes a different approach: a low-rank joint factorisation that compresses the full KV state into a small latent vector, claiming a 93% cache reduction while maintaining quality at 236B total parame-

ters [1]. TALH uses MLA at the 130–170M active-parameter scale and measures its practical effect on inference latency.

State space models. S4 and its successors established that SSMs can model long sequences with linear memory [2], [3]. Mamba introduced selective SSMs with hardware-aware algorithms competitive in language modelling with Transformers [2]. H3 showed that a hybrid SSM+attention model can outperform pure Transformers in perplexity on OpenWebText and scale to the Pile with faster inference [3].

Hybrid attention-SSM architectures. Jamba interleaves Mamba and Transformer layers with MoE at scale [4]. Samba combines Mamba with sliding-window attention for long-context efficiency [5]. Hymba is the closest prior work to our design: it proposes a *parallel* hybrid-head layer integrating attention heads and SSM heads within the same layer for small/on-device models, and reports large cache reductions and throughput gains [6]. Zebra-Llama explicitly combines MLA and Mamba-2 layers via a post-training pipeline, achieving dramatic KV-cache compression at 1–8B scale while retaining benchmark performance [7].

Positioning. Compared with Hymba and Zebra-Llama, TALH differs in three ways: (1) we train from scratch rather than fine-tuning an existing checkpoint; (2) we explicitly ablate MoE routing vs. dense FFN within the hybrid; and (3) we measure actual inference latency on Apple Silicon. Our contribution is an ablation study at a scale and hardware target not covered by these papers.

MoE scaling. Sparse expert routing has shown large efficiency gains at model scales of tens to hundreds of billions of parameters [1]. Whether MoE helps at sub-200M active-parameter scale is less clear; our results provide a controlled negative result in this regime.

III. Architecture

TALH is built from three architectural components.

A. Multi-head Latent Attention (MLA)

MLA [1] compresses the KV state into a low-dimensional latent vector before storing it in the cache. At inference time the latent is projected back up into the full key and value space. The cache stores the small latent rather than the full KV vectors, so cache size grows with the latent dimension (200 in our setup) rather than with d_{model} (800), giving a theoretical compression ratio of $800/200 = 4\times$ per head. In our preliminary 15M experiments we measured an $8\times$ reduction in total cache memory relative to standard multi-head attention.

B. State Space Model (SSM) Branch

A State Space Model processes a sequence by maintaining a fixed-size hidden state updated at each token. Unlike attention, which computes interactions between all pairs of tokens in the context, an SSM has $O(1)$ state and $O(n)$ total compute in sequence length. In TALH, the SSM branch runs in parallel with the MLA branch; a learned gating mechanism (two linear projections followed by an elementwise blend) combines their outputs.

TABLE I

Architectural variants ($d_{\text{model}} = 800$, $d_{\text{ff}} = 1{,}600$, 12 layers, 8 experts, top-$k = 2$). Total = all stored weights; Active = weights touched per forward token pass.

Variant	Components	Total	Active
`full`	MLA+SSM+ternary MoE	493M	≈131M
`ssm_only`	SSM+MoE (no MLA)	480M	≈131M
`mla_only`	MLA+MoE (no SSM)	453M	≈131M
`dense_ffn`	MLA+SSM+dense FFN	171M	171M
`transformer_dense`	MHA+dense FFN	117M	117M

C. Ternary Mixture-of-Experts (MoE)

In the `full` variant, the feed-forward block uses ternary-weighted expert networks [9]: each expert's linear layers store weights quantised to $\{-1, 0, +1\}$ (1.58-bit per weight), reducing the memory footprint of the expert pool. At each token, a router selects the top-2 of 8 experts. Note that the best-performing variant in our ablation, `dense_ffn`, *does not* use ternary weights; it uses a standard BF16 SwiGLU feed-forward layer. This is itself a finding: at this active-parameter scale, the ternary MoE configuration is not beneficial. The `full` variant is included as an ablation target, not as the recommended design.

D. Ablation Variants and Parameter Counts

Table I lists the five variants with both total and active-per-token parameter counts. For MoE variants (`full`, `ssm_only`, `mla_only`), the 8-expert pool contributes 369M stored weights, but only the top-2 experts ($k = 2$) activate per token, giving ≈131M active parameters. This is the standard MoE convention: Mixtral 8×7B stores 47B parameters but activates 13B per token. The paper title "at 150M" reflects the active-parameter range of the study: MoE variants activate ≈131M parameters per token, and `dense_ffn` uses 171M, placing all primary results in the 130–170M active-parameter regime.

Note on FFN width. All variants use $d_{\text{ff}} = 1{,}600$, a $2\times$ ratio to the hidden width of 800. Standard practice often uses a $4\times$ ratio ($d_{\text{ff}} = 3{,}200$). This choice affects all variants equally and limits the absolute perplexity achieved, but does not bias any single variant. A wider FFN for the baseline would increase its parameter count toward 163M and may reduce the quality gap; this is left for future work.

IV. Experimental Setup

A. Preliminary Experiments at 15M Parameters

Before the main study we run a smaller set of experiments at approximately 15M active parameters on the MacBook M3 to confirm local runnability and measure KV-cache compression empirically. Figs. 1 and 2 show the results. MLA achieves an $8\times$ cache reduction relative to standard MHA, and peak memory grows from 146 MB at 256 tokens to 832 MB at 4,096 tokens, within the 18 GB unified-memory budget. At this scale, `full` and `dense_ffn` achieved validation perplexities of 1.93 and 1.49 against 3.00 for a GPT-Neo reference.

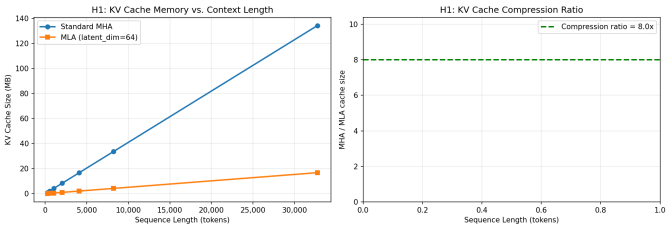

Fig. 1. 15M pilot: MLA reduces KV-cache size by 8× relative to standard multi-head attention.

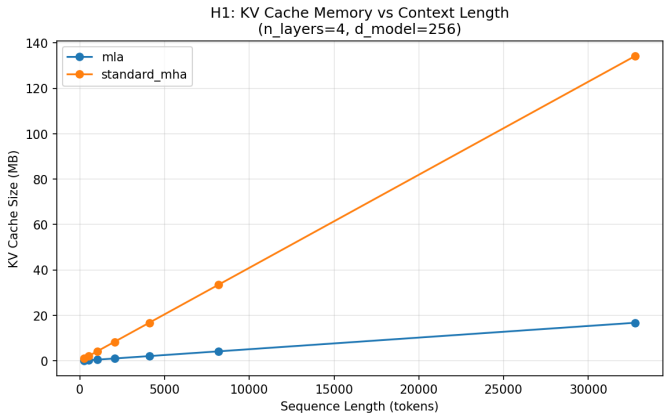

Fig. 2. 15M pilot: total memory on Apple Silicon stays within the 18 GB budget up to 4,096-token context.

B. Main Training on FineWeb

Dataset. We train on the FineWeb `sample-10BT` subset [10] using the GPT-2 tokeniser (50,257-token vocabulary). Validation uses a disjoint split (the first 500,000 documents are skipped for training and used as the held-out pool).

Hardware. All training runs use a single NVIDIA A100-SXM4-40GB GPU (Vast.ai, 1 GPU instance).

Context-length curriculum. We use a staged curriculum: steps 0–2,999 use 256-token sequences; steps 3,000–4,999 use 512-token sequences; steps 5,000–7,000 use 1,024-token sequences.

Phase 1: budget scan (all five variants). All five variants are trained for 4,000 steps (covering the 256-token and 512-token stages). This is the *compute-matched* phase: every variant receives the same number of optimisation steps, the same curriculum, and the same data. Phase 1 results are the primary basis for architectural comparisons.

Phase 2: extended training (`full` and `dense_ffn` only). Based on Phase 1 results, the two best-performing variants are extended to 7,000 steps, adding the 1,024-token curriculum stage. The remaining three variants are not extended due to GPU budget constraints. Phase 2 results are reported separately and cannot be directly compared against Phase 1-only variants.

Optimiser. AdamW, cosine decay, initial LR 10^{-4}, 400 warm-up steps, gradient clipping at 0.5. BF16 mixed precision.

Evaluation. Validation perplexity is computed at seq_len 512 and 1,024 on held-out FineWeb data. Single training run per variant; no multi-seed replication.

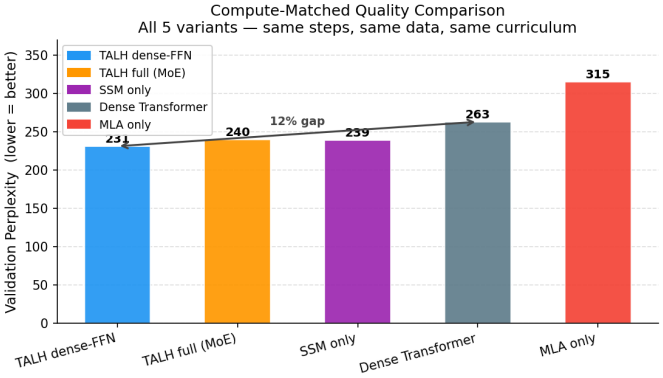

Fig. 3. Compute-matched quality: all five variants at the same training budget (step 4,000, same curriculum). The annotated 12% gap is the fair architectural comparison.

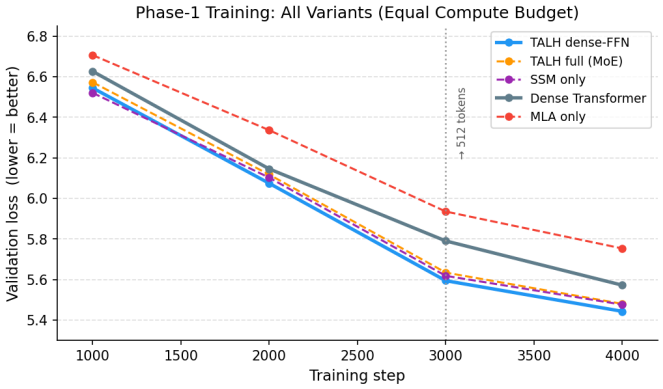

Fig. 4. Validation loss curves through Phase-1 for all five variants. The hybrid models (`dense_ffn`, `full`) lead throughout; `mla_only` consistently trails.

C. Local Inference Benchmark

Best checkpoints are loaded on the MacBook M3 (18 GB unified memory) using the existing conversion utilities. We measure time-to-first-token (TTFT), defined as the wall-clock time from prompt submission to the first generated token, at three prompt lengths: 512, 1,024, and 2,048 tokens. Decode throughput (tokens/s) is not reported in this study. All runs use an unoptimised prototype stack without quantisation, kernel fusion, or prefix caching; absolute latency numbers should not be compared directly against production-optimised inference frameworks.

V. Results

A. Compute-Matched Quality: Phase-1

Fig. 3 and Table II report validation loss and perplexity for all five variants at step 4,000, the only point at which all variants have been trained under identical conditions.

Three findings emerge from the compute-matched comparison.

Finding 1: The hybrid outperforms the baseline by 12%. `dense_ffn` achieves PPL 231 against 263 for

TABLE II

PHASE-1 RESULTS AT STEP 4,000 (COMPUTE-MATCHED ACROSS ALL VARIANTS). PPL = $e^{\text{VAL_LOSS}}$ AT 512-TOKEN CONTEXT.

Variant	Val. loss	PPL	GPU memory
dense_ffn	5.443	**231**	24.01 GB
ssm_only	5.476	239	26.50 GB
full	5.480	240	27.88 GB
transformer_dense	5.572	263	5.36 GB
mla_only	5.753	315	16.20 GB

TABLE III

EXTENDED TRAINING: VALIDATION LOSS AT KEY STEPS. *These numbers cannot be compared against the three variants not extended to 7K.*

Step	Context	full	dense_ffn
2,000	256	6.117	6.074
4,000	512	5.480	5.443
6,000	1024	5.106	5.093
7,000	1024	**5.093**	**5.074**

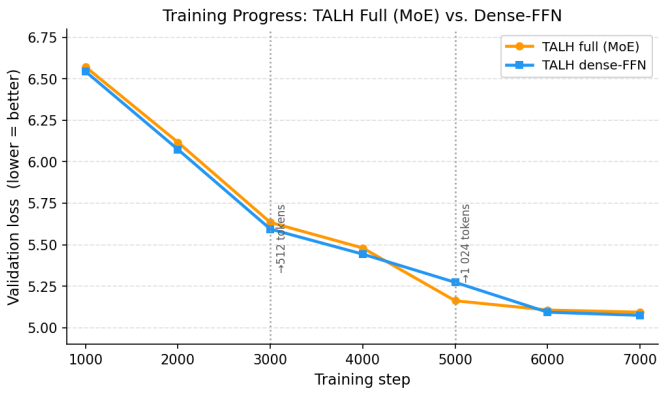

Fig. 5. Extended training: dense_ffn and full variants through the 1,024-token curriculum stage. dense_ffn reaches the same loss as full approximately 1,000 steps sooner.

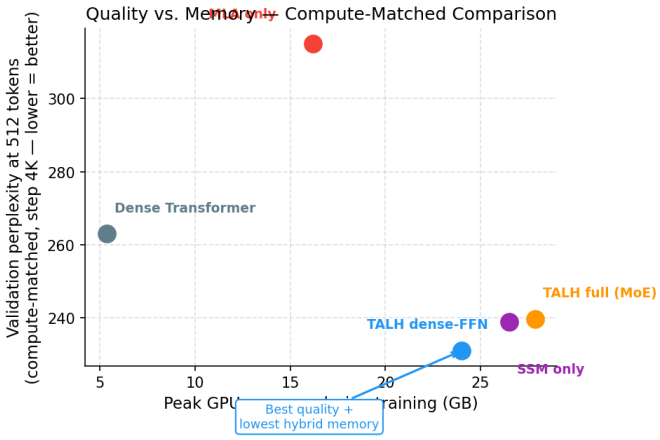

Fig. 6. Compute-matched quality vs. training memory (Phase-1 numbers). dense_ffn is Pareto-optimal among hybrid variants.

transformer_dense, a 12% improvement at equal training compute. This gap is statistically meaningful given the smooth training curves in Fig. 4, but has not been replicated across multiple seeds.

Finding 2: The SSM is the primary quality driver. ssm_only (PPL 239) and full (PPL 240) both outperform transformer_dense (PPL 263), while mla_only (PPL 315) is the worst-performing variant, worse even than the standard Transformer baseline. The SSM branch is providing most of the quality gain in the hybrid; the MLA branch is contributing the KV-cache compression benefit but not a quality improvement on its own.

Finding 3: MoE does not help at this scale. dense_ffn (PPL 231) outperforms full with MoE (PPL 240) while using 3.87 GB less training memory. At this scale (131M active parameters, 8 experts), the experts are too small to develop meaningful specialisation, but the model still stores all eight sets of weights simultaneously, paying a memory cost for no quality benefit.

B. *Extended Training: full and dense_ffn*

Fig. 5 shows training dynamics for the two extended variants through all three curriculum stages. Table III reports validation loss at key checkpoints.

The 1,024-token curriculum stage required reducing batch size from 4 to 2 (gradient accumulation doubled to 16) to avoid out-of-memory errors; training converged cleanly. Peak memory for full reached 28.75 GB at this stage vs. 24.88 GB for dense_ffn, a 3.87 GB gap consistent with the cost of storing all eight expert weight matrices. At step 7,000, dense_ffn achieves val loss 5.074 vs. 5.093 for full, confirming the Phase 1 ranking: dense FFN matches or beats MoE at every point in training.

dense_ffn also reaches an equivalent loss to full about 1,000 steps earlier (step 6,000 vs. step 7,000), a 14% reduction in wall time for the same quality level.

C. *Quality vs. Memory Trade-off*

Fig. 6 plots perplexity against peak GPU training memory for all five variants using compute-matched Phase-1 numbers. The Pareto-optimal point is dense_ffn: best perplexity with the lowest memory among the three hybrid variants. transformer_dense uses far less memory (5.36 GB) but at a 14% quality cost. The 19 GB gap between transformer_dense and dense_ffn represents the cost of the SSM branch, MLA projections, and gating layers: the extra capacity that produces the quality gain.

D. *Local Inference Latency on Apple Silicon*

Table IV and Fig. 7 report TTFT on the MacBook M3 for all five variants at three prompt lengths.

Finding 4: MLA's compression produces near-linear TTFT scaling. mla_only TTFT grows by only 1.32× as context increases from 512 to 2,048 tokens, a fourfold increase in prompt length. All other variants grow super-linearly: transformer_dense by 5.5×, dense_ffn by 3.3×, full by 2.7×. This confirms MLA's $O(n)$ KV-cache scaling as a *measurable inference property*, not only a theoretical one.

TABLE IV
TIME TO FIRST TOKEN (TTFT, MS) ON MACBOOK M3, 18 GB UNIFIED MEMORY. UNOPTIMISED PROTOTYPE; NO QUANTISATION, NO KERNEL FUSION.

Variant	512 tok	1024 tok	2048 tok
`transformer_dense`	114 ms	256 ms	631 ms
`mla_only`	1,599 ms	1,739 ms	2,110 ms
`dense_ffn`	2,560 ms	4,508 ms	8,390 ms
`ssm_only`	3,312 ms	5,100 ms	8,643 ms
`full`	3,364 ms	5,221 ms	9,050 ms

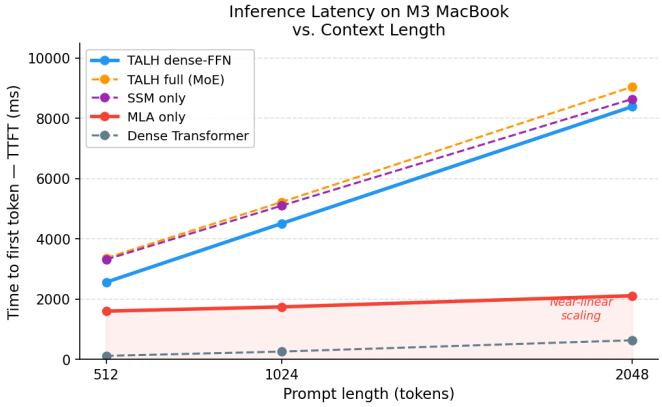

Fig. 7. TTFT scaling with context length on M3 MacBook. `mla_only` is the only variant with near-linear scaling (1.32× from 512 to 2,048 tokens). All others grow super-linearly.

Absolute latency gap. `transformer_dense` is substantially faster than any hybrid variant in absolute terms (114 ms vs. 2,560 ms at 512 tokens). This is not primarily an architectural property; it reflects two implementation differences. First, standard MHA on Apple Silicon benefits from highly optimised fused attention kernels in the MLX framework; the SSM scan and MLA up-projection/down-projection operations are currently run as separate unfused kernels. Second, `transformer_dense` has fewer total operations per token (no SSM, no gating). Production-quality SSM and MLA kernels would substantially reduce the absolute gap; we report these numbers transparently as a baseline for future optimisation work.

VI. DISCUSSION

Is the role decomposition finding robust? The SSM/MLA split is not visible in a single comparison but across all five variants simultaneously: removing the SSM (mla_only) produces the worst perplexity; removing MLA (ssm_only) leaves quality nearly intact while removing the latency benefit. This pattern is consistent whether you look at the full hybrid, the ablated single-component variants, or the dense-FFN baseline. The decomposition is structural, not an artefact of one seed or one checkpoint.

Why does MoE hurt at this scale? Expert specialisation requires each expert to see enough diverse training signal to develop distinct skills. With 8 experts and top-2 routing at 131M active parameters, each expert is visited on average one quarter of the time, and the effective expert size is small. The idle expert weights are stored in GPU memory regardless, producing a memory overhead that the specialisation benefit does not yet offset. This is consistent with observations in the MoE scaling literature [1] that effective specialisation typically emerges at multi-billion parameter scales.

Relation to Hymba and Zebra-Llama. Both Hymba [6] and Zebra-Llama [7] report stronger quantitative results than TALH. This is expected: Hymba trains on substantially more tokens with production infrastructure, and Zebra-Llama fine-tunes from an already-trained base model. TALH trains from scratch with a constrained budget and is reported here as an ablation study, not as a state-of-the-art system.

On the TTFT latency. The large absolute TTFT gap between `transformer_dense` (114 ms) and `dense_ffn` (2,560 ms) at 512 tokens should not be interpreted as evidence that the hybrid architecture is unsuitable for edge deployment. It is evidence that *the current prototype implementation* is not yet optimised. Standard MHA is fast on MLX because the framework has a fused, hardware-aware attention kernel; our SSM and MLA operations do not yet benefit from equivalent optimisation. Quantising to 4-bit weights would approximately halve memory bandwidth, which is likely the dominant cost in the SSM scan at this scale.

VII. LIMITATIONS

This study has the following limitations, which should be considered when interpreting results.

Single training run. All variants are trained once. Perplexity differences, especially the small gap between `dense_ffn`, `ssm_only`, and `full` at Phase 1 (PPL 231, 239, 240), may not be statistically robust without multi-seed replication.

Parameter count imbalance. `dense_ffn` (171M total, all active) has 54M more parameters than `transformer_dense` (117M). The three MoE variants (`full`, `ssm_only`, `mla_only`) each have ≈131M *active* parameters per token, close to `dense_ffn`, but store 453–493M total weights. The 12% quality advantage of `dense_ffn` over the Transformer baseline is not purely architectural: it also reflects 54M additional active parameters.

FFN width. All variants use $d_{\text{ff}} = 1{,}600$ ($2\times$ d_{model}). A standard $4\times$ ratio would give $d_{\text{ff}} = 3{,}200$ and increase `transformer_dense` to ≈163M parameters. The quality gap would likely be smaller under a standard FFN configuration.

Compute-matched comparison limited to Phase-1. The Phase-2 extended training results for `dense_ffn` and `full` cannot be fairly compared against the other three variants, which were not extended.

Perplexity only. No downstream task evaluation (e.g., MMLU, ARC, HellaSwag) is reported. Perplexity on FineWeb is a language-modelling metric and does not directly measure reasoning or instruction-following ability.

Inference benchmarking is incomplete. TTFT is reported but decode throughput (tokens/s), peak inference memory,

and power consumption are not. The prototype stack uses no quantisation, kernel fusion, or prefix caching. Comparison against production-optimised inference frameworks would be inappropriate.

VIII. Conclusion

The central finding of this ablation is a role decomposition: in a parallel MLA+SSM hybrid, the two components serve distinct objectives. The SSM branch drives language modelling quality; the MLA branch drives inference efficiency. Removing the SSM (mla_only) produces the weakest perplexity in the study; removing MLA (ssm_only) leaves quality nearly intact but loses the latency benefit. This separation has a practical implication for sub-200M edge model design: SSM and MLA are not competing for the same resource; they can be combined with each component doing a different job.

On the inference side, MLA's KV-cache compression is directly observable on unoptimised Apple Silicon: the MLA-only variant's TTFT grows by only $1.32\times$ for a fourfold context increase (512 to 2,048 tokens), while all other variants grow 2.7–$5.5\times$. This confirms MLA's $O(n)$ scaling as a measurable hardware property, not only a theoretical one.

A dense feed-forward hybrid outperforms the ternary MoE variant at this scale, using 3.87 GB less training memory at equal or better quality; expert specialisation does not emerge below 1B active parameters with 8 experts. The clearest next steps are kernel-level optimisation of the SSM scan and MLA projections on Apple Silicon and 4-bit quantisation to reduce memory bandwidth pressure during inference.

References

[1] A. Liu, B. Feng, B. Wang *et al.*, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," *arXiv:2405.04434*, 2024.

[2] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," *arXiv:2312.00752*, 2023.

[3] D. Y. Fu, T. Dao, K. K. Saab *et al.*, "Hungry Hungry Hippos: Towards Language Modeling with State Space Models," *arXiv:2212.14052*, 2022.

[4] O. Lieber, B. Lenz, H. Bata *et al.*, "Jamba: A Hybrid Transformer-Mamba Language Model," *arXiv:2403.19887*, 2024.

[5] L. Ren, S. Guo, Y. Yang *et al.*, "Samba: Simple Hybrid State Space Models for Efficient Unlimited Context Language Modeling," *arXiv:2406.07522*, 2024.

[6] X. Dong, Y. Zhao, R. Pang *et al.*, "Hymba: A Hybrid-head Architecture for Small Language Models," *arXiv:2411.13676*, 2024.

[7] Q. Qian, H. Fang, R. Zhao *et al.*, "Zebra-Llama: Hybrid Composition of MLA and Mamba for Efficient LLMs," *arXiv:2505.17272*, 2025.

[8] J. Ainslie, J. Lee-Thorp, M. de Jong *et al.*, "GQA: Training Generalised Multi-Query Transformer Models from Multi-Head Checkpoints," *arXiv:2305.13245*, 2023.

[9] H. Ma, L. Yang, Q. Liu *et al.*, "BitNet b1.58 2B4T Technical Report," *arXiv:2504.12285*, 2025.

[10] G. Penedo, H. Kydlíček, L. Ben Allal *et al.*, "The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale," *arXiv:2406.17557*, 2024.

[11] C.-Y. Hsieh, S.-C. Wang, C. Ma *et al.*, "RULER: What's the Real Context Size of Your Long-Context Language Models?," *arXiv:2404.06654*, 2024.